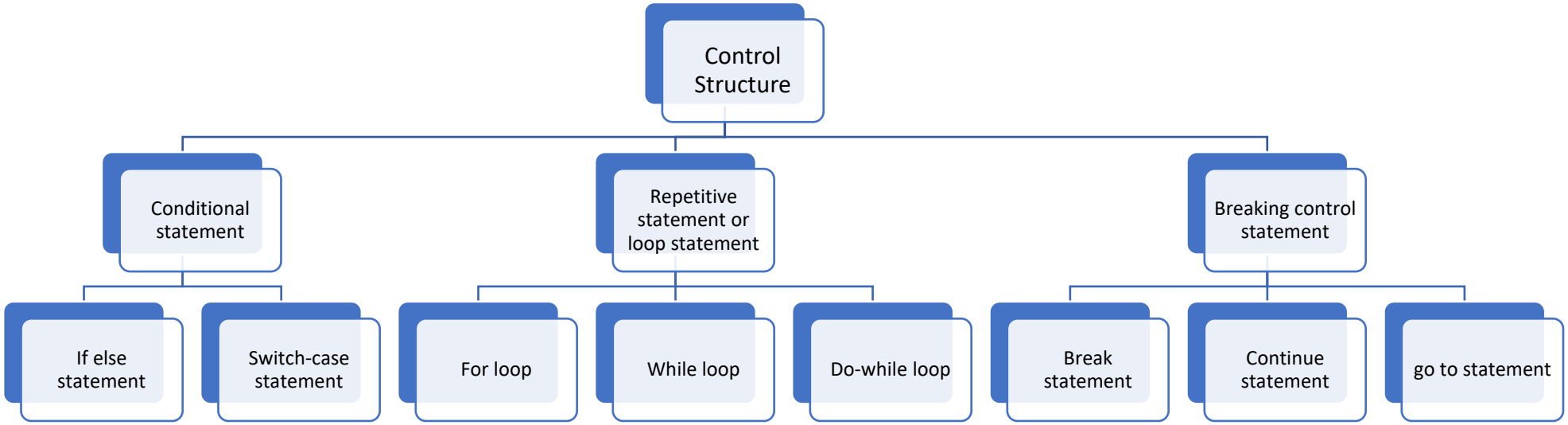
Unit- 1

Control Structure

Control Structure

- Depending upon requirements of a problem, it is often required to alter the normal sequence of execution in the program. This means that we may desire to selectively or repetitively execute a program segment



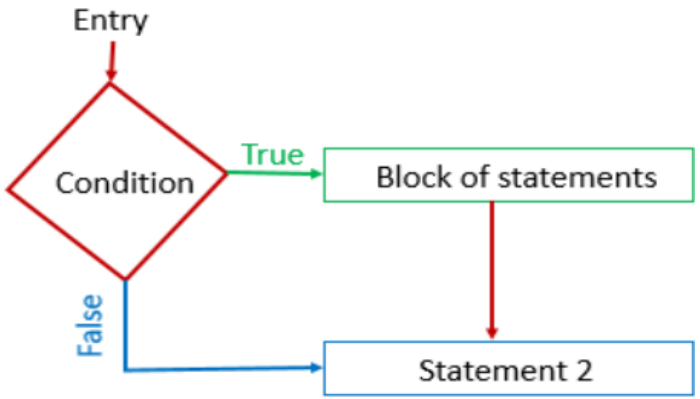
Conditional statement

- Conditional statements also known as decision statements are mainly used for making decisions
- Depending on the value of an expression, decision can be made. If a condition evaluates to true, a set of statements are executed otherwise different set of statements are executed. Following are the conditional statements in C/C++:
  - if statement
  - if else statement
  - nested if else
  - if-else-if ladder
  - switch statement

Conditional statement

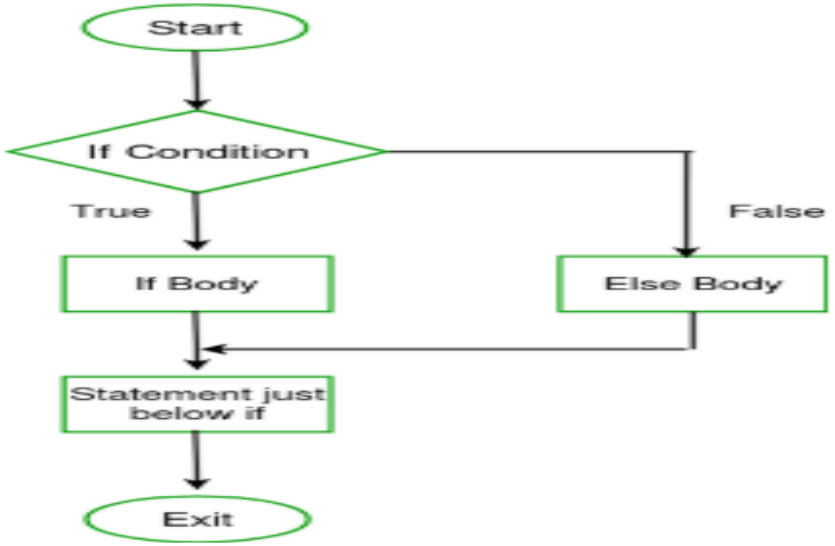
- **If statement:** The syntax is:

```
if (expression)
        {
          Body of if
        }
```



- **If else statement:** The syntax is:

```
 if (expression)
        {
          Body of if
        }
   else
         {
          Body of else
        }
```



```c
#include <stdio.h>
int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);

    // true if num is perfectly divisible by 2
    if(num % 2 == 0)
        printf("%d is even.", num);
    else
        printf("%d is odd.", num);

    return 0;
}
```

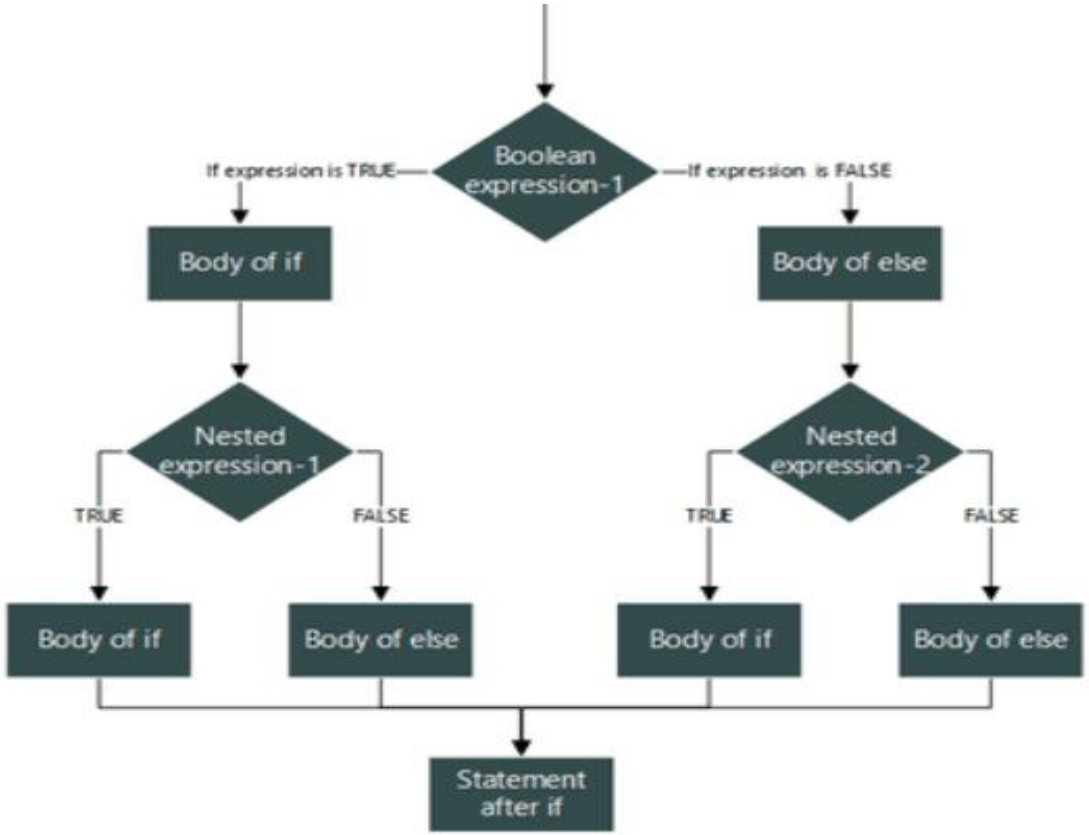
Output

```
Enter an integer: -7
-7 is odd.
```

Conditional statement

- **Nested if-else:** In **if** statement of **if-else** statement in place of body we can again use **if** or **if-else**. This concept is known as nested structure. Nesting of if statements is very helpful when you have something to do by following more than one decision
- The syntax is:

```
 if (expression1)
        {
          if(expression)
             {
              }
          else
             {
              }
         }
else
       {
          if(expression)
             {
              }
          else
             {
              }
         }
```



Conditional statement

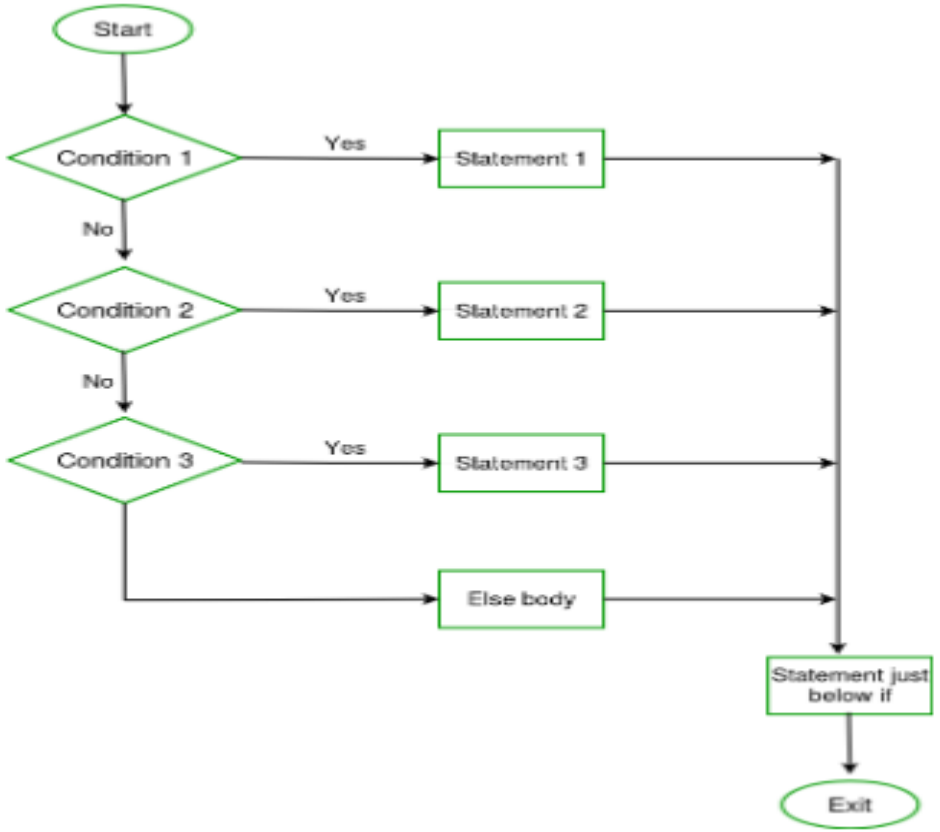
// Program to illustrate use of nested if-else:

```
#include<stdio.h>
int main()
{
	int num=1;
	if(num<10)
	{
		if(num==1)
		{
			printf("The value is:%d\n",num);
		}
		else
		{
			printf("The value is greater than 1");
		}
	}
	else
	{
		printf("The value is greater than 10");
	}
	return 0;
}
```

Conditional statement

- **if-else-if ladder:** It helps user decide from among multiple options. The C/C++ if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C else-if ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.
- **Syntax:**

```
if (condition1)
   statement 1;
else if (condition2)
   statement 2;
.
.
else
   statement;
```



Conditional statement

// Program to illustrate use of if-else-if ladder:

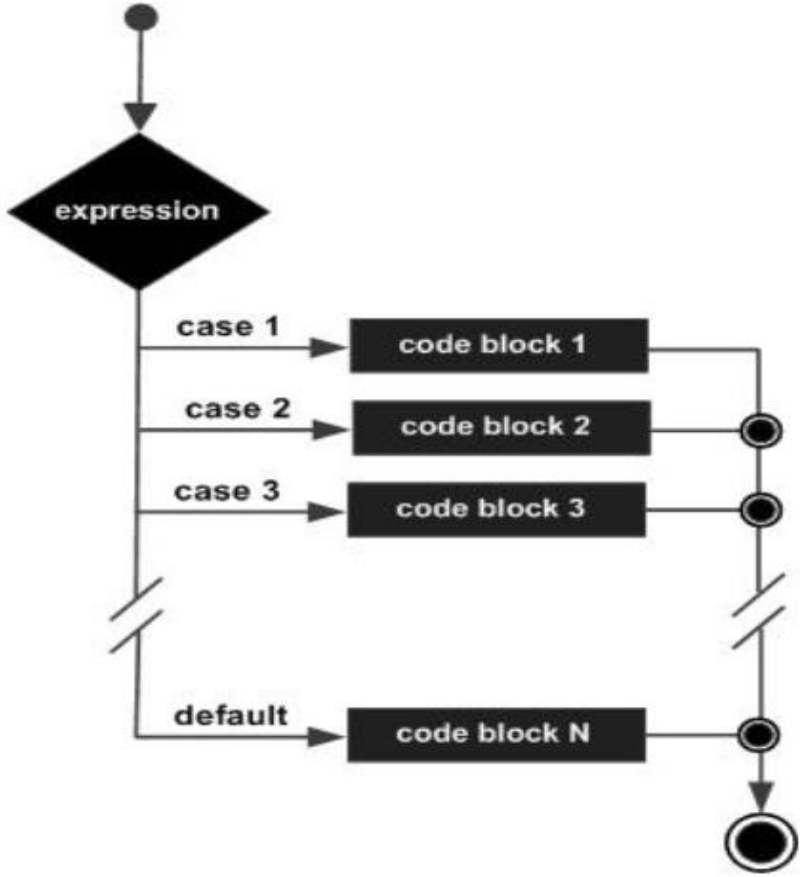
```
// C Program to Calculate Grade According to marks
// using the if else if ladder
#include <stdio.h>
int main()
{
    int marks = 91;
    if (marks <= 100 && marks >= 90)
        printf("A+ Grade");
    else if (marks < 90 && marks >= 80)
        printf("A Grade");
    else if (marks < 80 && marks >= 70)
        printf("B Grade");
    else if (marks < 70 && marks >= 60)
        printf("C Grade");
    else if (marks < 60 && marks >= 50)
        printf("D Grade");
    else
        printf("F Failed");
    return 0;
}
```

Conditional statement

- **Switch statement:** Like other conditional statements, ‘switch’ is a type of selection control mechanism which allows the value of a variable or expression to be tested for equality against a list of constant expressions (each one is called a case), thus transferring the control flow to the matched case.

```
switch (Variable)
{
      case <value_1> :   statement1;
                         statement2;
                         statement3;
                         break;

      case <value_2> :   statement1;
                         statement2;
                         break;

      case <value_3> :   statement1;
                         break;

      default:           statement1;
                         statement2;
}
```

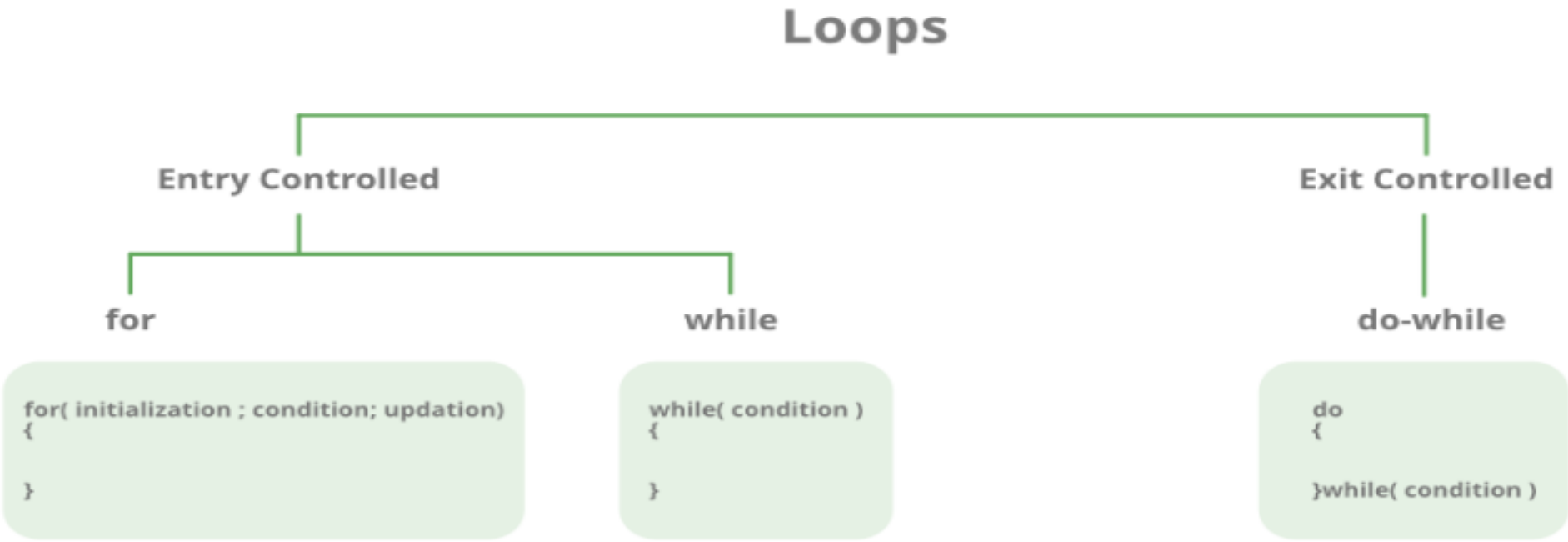
Optional




```c
#include <stdio.h>
int main () {
   char grade = 'B';
   switch(grade) {
      case 'A' :
         printf("Excellent!\n" );
         break;
      case 'B' :
      case 'C' :
         printf("Well done\n" );
         break;
      case 'D' :
         printf("You passed\n" );
         break;
      case 'F' :
         printf("Better try again\n" );
         break;
      default :
         printf("Invalid grade\n" );
   }
   printf("Your grade is  %c\n", grade );
   return 0;
}
```

Repetitive statement or Loop statement

- Loops are used to execute a set of statements repeatedly until a particular condition is satisfied
- A looping process, in general include the following steps:
  - Initialization of a counter
  - Execution of statements in the loop
  - Test for a specified condition for execution of the loop
  - Incrementing/Decrementing the counter

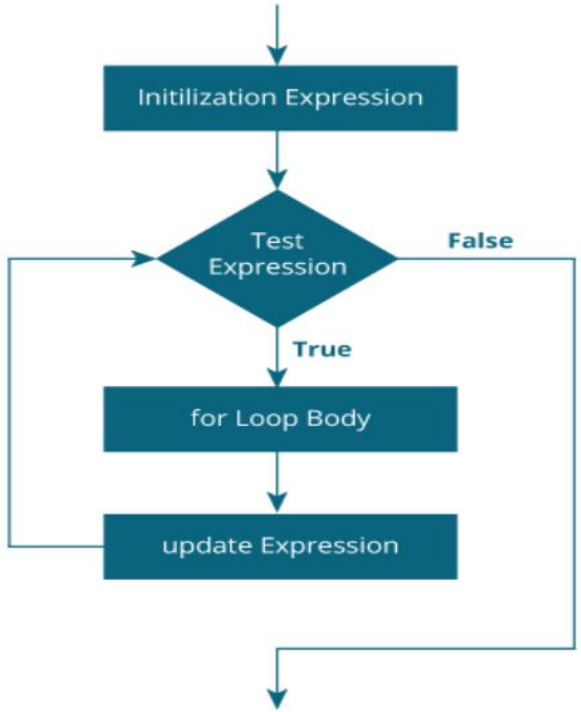


For Loop

- for loop is used to execute a set of statements repeatedly until a particular condition is satisfied. It is also known as entry controlled loop. General format is:

```
for(initialization; condition; increment/decrement)
{
    statement-block;
}
```

- In for loop we have exactly two semicolons, one after initialization and second after the condition. In this loop we can have more than one initialization or increment/decrement, separated using comma operator. But it can have only one condition



For Loop

In C/C++, the for loop can have multiple expressions separated by commas in each part:

```
for (x = 0, y = num; x < y; i++, y--) {
  statements;
}
```

Also, we can skip the initial value expression, condition and/or increment by adding a semicolon.

For example:

```
int i=0;
int max = 10;
for (; i < max; i++) {
  printf("%d\n", i);
}
```

A loop becomes an infinite loop if a condition never becomes false. You can make an endless loop by leaving the conditional expression empty.

```
#include <stdio.h>

int main () {

   for( ; ; ) {
      printf("This loop will run forever.\n");
   }

   return 0;
}
```

When the conditional expression is absent, it is assumed to be true. You may have an initialization and increment expression but more commonly used construct is for(;;) to signify an infinite loop.

While Loop

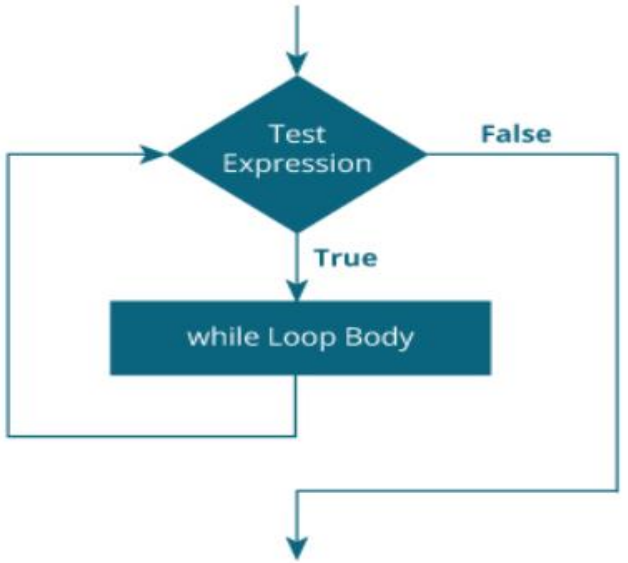
- The **while loop in C/C++** is used in situations where we do not know the exact number of iterations of loop beforehand. The loop execution is terminated on the basis of the test condition.
- The syntax of the while loop:

```
while (testExpression)
{
    // statements inside the body of the loop
}
```

```
// Print numbers from 1 to 5

#include <stdio.h>
int main()
{
      int i = 1;

      while (i <= 5)
      {
            printf("%d\n", i);
            ++i;
      }

      return 0;
}
```

Output

```
1
2
3
4
5
```



do..while Loop

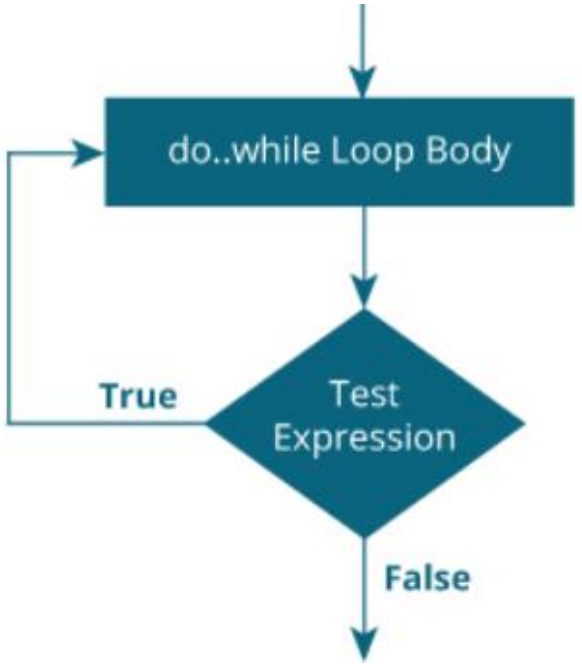
- The do..while loop is similar to the while loop with one important difference. The body of do...while loop is executed at least once. Only then, the test expression is evaluated.
- The syntax of the do...while loop is:

```
do
{
   // statements inside the body of the loop
}
while (testExpression);
```

```
// Program to add numbers until the user enters zero

#include <stdio.h>
int main()
{
    double number, sum = 0;

    // the body of the loop is executed at least once
    do
    {
        printf("Enter a number: ");
        scanf("%lf", &number);
        sum += number;
    }
    while(number != 0.0);

    printf("Sum = %.2lf",sum);

    return 0;
}
```

Output

```
Enter a number: 1.5
Enter a number: 2.4
Enter a number: -3.4
Enter a number: 4.2
Enter a number: 0
Sum = 4.70
```



Breaking control statements

- The **break** statement in C programming has the following two usages:
  - When a **break** statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
  - It can be used to terminate a case in the **switch** statement (covered in the next chapter).
- If you are using nested loops, the break statement will stop the execution of the innermost loop and start executing the next line of code after the block.

```
while (testExpression) {
   // codes
   if (condition to break) {
      break;
   }
   // codes
}
```

```
do {
   // codes
   if (condition to break) {
      break;
   }
   // codes
}
while (testExpression);
```

```
for (init; testExpression; update) {
   // codes
   if (condition to break) {
      break;
   }
   // codes
}
```

```
#include <stdio.h>
int main()
{
    int i,j;
    for (i = 0; i < 5; i++)
    {
        for (j = 1; j <= 10; j++) {
            if (j > 3)
                break;
            else
                printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePauser.exe
***
***
***
***
***

--------------------------------
Process exited with return value 0
Press any key to continue . . .
```

Breaking control statements

- The continue statement skips the current iteration of the loop and continues with the next iteration

```
while (testExpression) {
    // codes
    if (testExpression) {
        continue;
    }
    // codes
}
```

```
do {
    // codes
    if (testExpression) {
        continue;
    }
    // codes
} while (testExpression);
```

```
for (init; testExpression; update) {
    // codes
    if (testExpression) {
        continue;
    }
    // codes
}
```

```c
// Program to calculate the sum of a maximum of 10 numbers
// Negative numbers are skipped from the calculation
# include <stdio.h>
int main()
{
    int i;
    double number, sum = 0.0;
    for(i=1; i <= 10; ++i)
    {
        printf("Enter a n%d: ",i);
        scanf("%lf",&number);
        if(number < 0.0)
        {
            continue;
        }
        sum += number;
    }
    printf("Sum = %.2lf",sum);
    return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\Cc
Enter a n1: 1.1
Enter a n2: 2.2
Enter a n3: 5.5
Enter a n4: 4.4
Enter a n5: -3.4
Enter a n6: -45.5
Enter a n7: 34.5
Enter a n8: -4.2
Enter a n9: -1000
Enter a n10: 12
Sum = 59.70
--------------------------------
Process exited with return va
Press any key to continue . .
```

Breaking control statements

- The goto statement allows to transfer control of the program to the specified label.
- The label is an identifier. When the goto statement is encountered, the control of the program jumps to label: and starts executing the code.


```c
// Program to calculate the sum and average of positive numbers
// If the user enters a negative number, the sum and average are displayed.
#include <stdio.h>
int main()
{
    int maxInput = 100;
    int i;
    double number, average, sum = 0.0;
    for (i = 1; i <= maxInput; ++i) {
        printf("%d. Enter a number: ", i);
        scanf("%lf", &number);
        // go to jump if the user enters a negative number
        if (number < 0.0) {
            goto jump;
        }
        sum += number;
    }

jump:
    average = sum / (i - 1);
    printf("Sum = %.2f\n", sum);
    printf("Average = %.2f", average);
    return 0;
}
```

```
C:\Program Files (x86)\Dev-Cpp\ConsolePaus
1. Enter a number: 3
2. Enter a number: 4.3
3. Enter a number: 9.3
4. Enter a number: -2.9
Sum = 16.60
Average = 5.53
--------------------------------
Process exited with return value 0
Press any key to continue . . .
```